DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Migración de Script y comunicación entre Microservicios

DESARROLLO FULLSTACK I

DSY1103

FORMAMOS PERSONAS PARA UNA SOCIEDAD MEJOR

• Contenidos

- Persistencia y Migraciones
- Comunicación Entre Servicios.
- Desarrollo de actividades practicas.

01

Persistencia y Migraciones

• Entidades y BD Independientes

El Reto de la Distribución

La arquitectura de microservicios distribuye la lógica de negocio, introduciendo desafíos en la gestión de datos y la comunicación entre servicios.

Principio Fundamental

"Base de datos por Microservicio". Cada microservicio es el único propietario de su esquema de base de datos. Se prohíbe el acceso directo a la BD de otro servicio.

Ventajas Clave

- **Aislamiento**: Fallos de BD quedan contenidos.
- **Escalabilidad**: Independiente para cada servicio.
- **Flexibilidad**: Elección de tecnología de BD por microservicio.

• Dependencias Clave para la Persistencia

JPA y Driver: Para la capa de datos.

Ejemplo: `spring-boot-starter-data-jpa` y el driver específico de la BD (ej. `postgresql` o `mysql`).

Migraciones (Flyway o Liquibase): Para gestionar el esquema de la BD.

Añadir `org.flywaydb:flyway-core` o `org.liquibase:liquibase-core` en tu `pom.xml`.

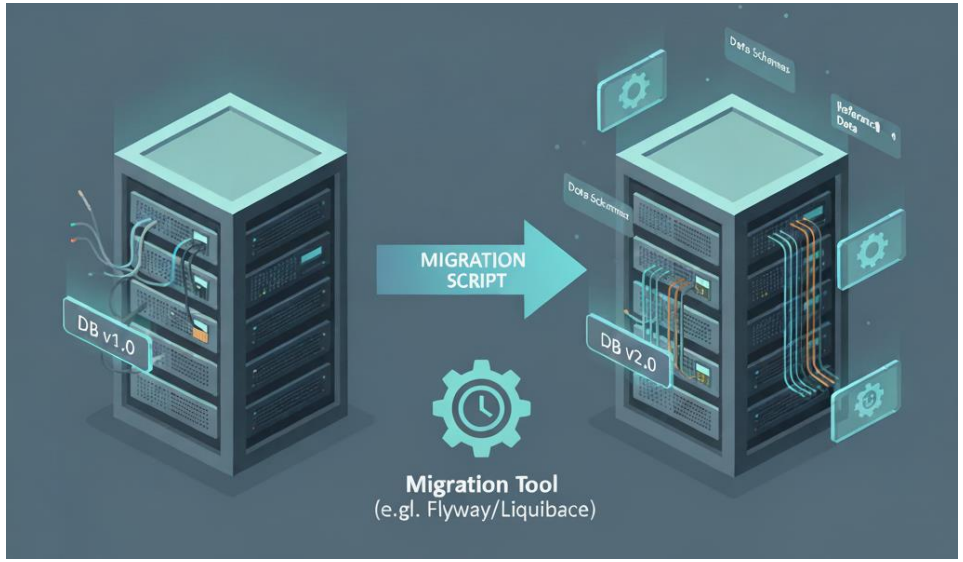
Configuración: Define las credenciales de la BD en `application.properties` para que Spring Boot y Flyway se conecten.

• ¿Qué son las migraciones de base de datos?

Las migraciones son archivos de instrucciones escritas en SQL que permiten:

- Crear tablas
- Modificar columnas
- Agregar relaciones
- Cargar datos iniciales
- Versionar cambios en la estructura

Son fundamentales en proyectos de microservicios porque permiten controlar la evolución de la base de datos de forma segura y reproducible.



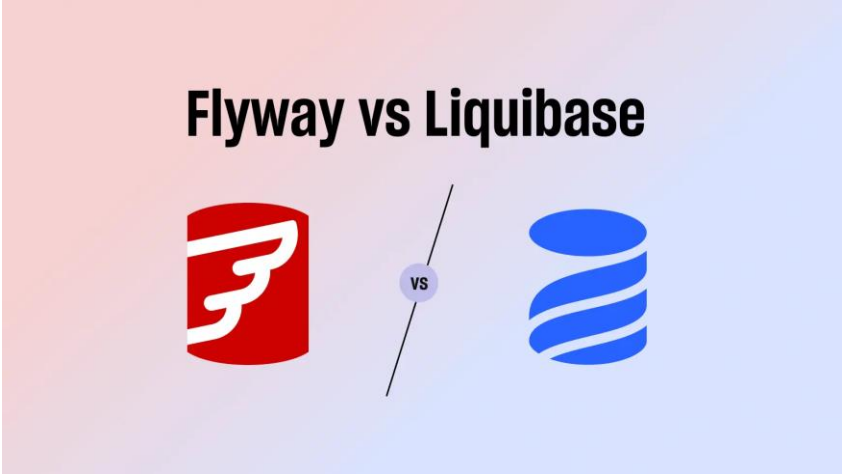
• ¿Por qué NO usar spring.jpa.hibernate.ddl-auto en proyectos reales?

Hibernate tiene modos como **update, create, create-drop**, que generan automáticamente las tablas.

Pero en producción:

- No lleva control de versiones
- Puede borrar tablas por error
- No es compatible con ambientes de despliegue CI/CD
- Cambios manuales no se registran
- No funciona bien cuando muchos desarrolladores trabajan en el mismo proyecto

Por eso se recomienda migrar hacia scripts SQL manuales o herramientas como Flyway / Liquibase.



• Migraciones de BD con Scripts Iniciales

Configuración y Ejecución

Los scripts son archivos de texto (SQL o XML), no código Java Spring. Estos archivos definen la estructura y datos iniciales (seed data).

- **Ubicación (Flyway):** src/main/resources/db/migration.
- **Ubicación (Liquibase):** El archivo principal (changelog) se suele ubicar en src/main/resources/db/changelog.
- **Automatización:** Spring Boot detecta automáticamente la librería y ejecuta todos los scripts pendientes al arrancar el microservicio.

• Migraciones de BD con Scripts Iniciales

Buenas prácticas al versionar scripts

- Usar nombres como V1__init.sql, V2__add_table_cliente.sql.
- Nunca modificar un script ya ejecutado: crear uno nuevo.
- Mantener SQL claro y compatible con el motor de BD.
- Cada microservicio debe tener sus propios scripts, nunca compartidos.

Estructura de un script inicial SQL

```sql
CREATE TABLE productos (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    precio DOUBLE NOT NULL,
    stock INT DEFAULT 0
);
```

Script con datos iniciales

```sql
INSERT INTO productos (nombre, precio, stock)
VALUES
('Teclado Gamer Redragon', 24990, 15),
('Monitor Samsung 24"', 129990, 8);
```

- **Documentación**

Flyway

Video de Apoyo

Liquibase

Documentación Oficial Liquibase

Video de Apoyo

02

Comunicación Entre Servicios

• Flujo de Comunicación con Datos Separados

La "Regla de Oro"

Si el "Servicio A" necesita datos del "Servicio B", nunca debe acceder directamente a la base de datos del Servicio B. Debe ser a través de una API.

- **Síncrona**: Llamada directa REST (WebClient/Feign). Bloquea al cliente hasta recibir respuesta.
- **Asíncrona**: Comunicación basada en eventos (Kafka, RabbitMQ). No bloquea al cliente.

• DTOs para comunicación

Cada microservicio debe exponer datos mínimos, ideales para consumo externo.

¿Por qué usar DTO y no enviar la entidad completa?

- Para evitar exponer estructura interna
- Para proteger campos sensibles
- Para mantener independencia entre microservicios
- Para controlar el tamaño de la respuesta

• COMUNICACIÓN ENTRE MICROSERVICIOS CON REST (WEBCLIENT / FEIGN)

¿Qué es WebClient?

Es un cliente HTTP asíncrono incluido con Spring WebFlux.

Permite realizar llamadas a otros servicios REST.

Ventajas:

- Muy flexible
- Soporta programación reactiva
- Ideal cuando necesitas alto rendimiento

Desventajas:

- Más detallado que Feign
- Más código

```
@Bean
public WebClient webClient() {
    return WebClient.builder().build();
}
```

```
public ProductoDTO obtenerProducto(Long id) {
    return webClient.get()
            .uri("http://localhost:8081/api/productos/" + id)
            .retrieve()
            .bodyToMono(ProductoDTO.class)
            .block();
}
```

• COMUNICACIÓN ENTRE MICROSERVICIOS CON REST (WEBCLIENT / FEIGN)

¿Qué es Feign Client?

Es un cliente declarativo para consumir microservicios REST, usado en Spring Cloud.

Permite crear una interfaz “como si fuera un Repository”, pero para llamadas externas.

Ventajas:

- Menos código
- Muy fácil de leer
- Perfecto para microservicios
- Maneja JSON automáticamente

Desventajas:

- Necesita Spring Cloud OpenFeign

```java
@EnableFeignClients
@SpringBootApplication
public class PedidoApplication {}
```

```java
@FeignClient(name = "productos", url = "http://localhost:8081/api/productos")
public interface ProductoClient {

    @GetMapping("/{id}")
    ProductoDTO getById(@PathVariable Long id);
}
```

```java
@Service
public class PedidoService {

    @Autowired
    private ProductoClient productoClient;

    public ProductoDTO validarProducto(Long id) {
        return productoClient.getById(id);
    }
}
```

• Comparación WebClient vs Feign

Característica	WebClient (Spring WebFlux)	Feign Client (Spring Cloud)
Paradigma	Reactivo (No Bloqueante)	Bloqueante (por defecto)
Estilo de Código	Programático (Fluent API)	Declarativo (Mediante una Interfaz Java)
Rendimiento/Hilos	Alto rendimiento, usa pocos hilos (Event-Loop)	Menor rendimiento en alta concurrencia, usa un hilo por petición
Dependencia	spring-boot-starter-webflux	spring-cloud-starter-openfeign
Curva de Aprendizaje	Moderada (Requiere entender conceptos reactivos)	Baja (Muy simple, se siente como una llamada local)
Caso de Uso Ideal	Microservicios de alta concurrencia, API Gateways, I/O intensivo	Comunicación simple entre servicios internos, rapidez de desarrollo

- **Documentación**

Flyway

Video de Apoyo

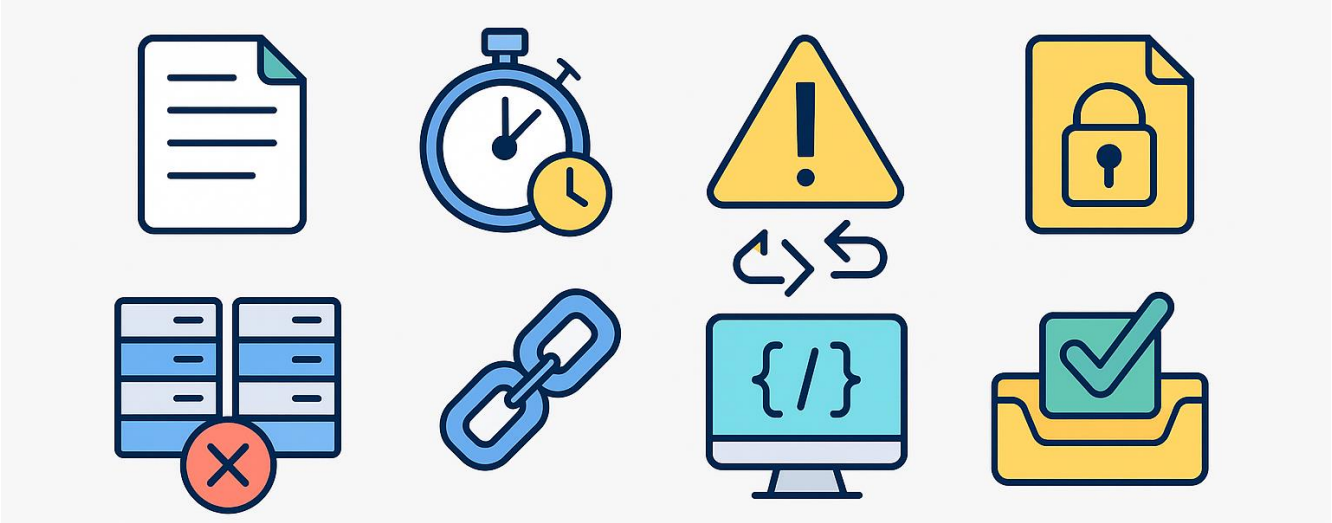
Liquibase

Documentación Oficial Liquibase

Video de Apoyo

• Buenas prácticas al comunicar microservicios

- Usar DTO claros y ligeros
- Manejar timeouts
- Manejar excepciones de comunicación
- Usar logs para trazabilidad entre servicios
- No exponer entidades completas
- Nunca compartir la misma base de datos
- Versionar contratos de API



• ACTIVIDAD PRÁCTICA 1

Usa el proyecto **Proyecto Hospital Vida y Meditación**

Integra script de migraciones de datos en una de las tablas que consideres necesaria.

• ACTIVIDAD PRÁCTICA 2

Implementa en el **Proyecto Hospital Vida y Meditación**

Divide en al menos dos microservicios mas pequeños para separar sus tablas y crear comunicación entre ellos

03

¡MUCHAS GRACIAS!

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.